TerraPulse: Bridging the Gap Between Code Quality and Security in IaC Ecosystems

A Longitudinal Empirical Study of Terraform Repositories

Francis Mascia
Dept. of Computer Engineering
University of Sannio
Benevento, Italy
f.mascia2@studenti.unisannio.it

Alfonso Maria Turco
Dept. of Computer Engineering
University of Sannio
Benevento, Italy
a.turco@studenti.unisannio.it

***Abstract*—The widespread adoption of Infrastructure as Code (IaC), with Terraform as its de facto standard, has shifted infrastructure management into a software engineering discipline, inheriting its benefits alongside its pathologies, such as technical debt. While the impact of code quality on maintainability is well-studied, its longitudinal relationship with security vulnerabilities in IaC remains largely unexplored. This paper introduces TerraPulse, a framework designed to conduct a large-scale empirical study on the co-evolution of structural quality and security in the Terraform ecosystem.**

By mining the complete version history of a statistically representative sample of 500 open-source repositories, we analyzed 10,054 historical snapshots, filtered to 9,259 unique code states to prevent temporal autocorrelation. From this data, we define and track two novel indices: a data-driven Structural Debt Index (StDI), with metrics weighted by their empirically derived predictive power, and a Security Debt Index (SDI), based on vulnerability scans.

Our findings indicate that structural metrics carry a meaningful predictive signal for security risk. A Random Forest model achieves a cross-validated R^2 of 0.397 in a repository-agnostic setup (log-transformed space), though generalization to fully held-out repositories on the original scale yields $R^2 = 0.133$, reflecting the high inter-repository heterogeneity typical of open-source IaC. The number of managed resources (`num_resources`) emerges as the dominant predictor (38% importance). We also uncover a "Complexity Paradox": although higher cyclomatic complexity is weakly positively associated with absolute security debt ($\rho = +0.054$), it is negatively correlated with debt density ($\rho = -0.319$) and shows a strong protective coefficient in multivariate regression (RLM: -251.29), suggesting that logical abstraction mitigates per-unit risk. Longitudinal analysis of 308 mature repositories reveals a systemic drift towards structural decay (74.4% of projects), with 31.5% exhibiting joint increases in both structural and security debt. Joint improvements are statistically rare (0.6%), consistent with the "sticky" nature of technical debt. Unsupervised clustering identifies three distinct project archetypes: *Low-to-Moderate-Debt* (28.4%), *High-Debt* (64.6%, dominated by infrastructure misconfigurations), and *Very-High-Debt* (7.0%, dominated by vulnerable dependencies).

This study provides empirical evidence that structural quality is associated with security posture in IaC, laying the groundwork for data-driven early-warning approaches in DevSecOps pipelines.

***Index Terms*—Infrastructure as Code (IaC), Terraform, Mining Software Repositories (MSR), Technical Debt, Security Debt, Longitudinal Analysis, Empirical Software Engineering.**

I. Introduction

The rise of Infrastructure as Code (IaC) has become a cornerstone of modern DevOps, radically transforming how cloud computing environments are provisioned and managed. IaC enables the automated, scalable, and reproducible management of infrastructure resources by defining them in machine-readable files. This paradigm treats infrastructure configurations—servers, networks, and databases—as software, subjecting them to the same rigorous engineering practices as application code, such as version control and peer review.

Within this domain, HashiCorp's Terraform has emerged as the de facto industry standard. It utilizes a proprietary declarative language, HCL (HashiCorp Configuration Language), allowing engineers to define the "desired state" of their infrastructure. Terraform's engine then computes the necessary operations to align the current state with the desired one. Its provider-based flexibility and widespread adoption make it an ideal subject for an in-depth study on the evolution of modern cloud architectures.

With this shift, IaC has inherited not only the benefits of traditional software engineering but also its intrinsic pathologies, most notably Technical Debt [1]. This metaphor, originally coined by Ward Cunningham, describes the long-term consequences of suboptimal design choices made to prioritize delivery speed over code quality. In the context of IaC, technical debt manifests in two primary forms:

- **Structural Quality Debt**: Violations of design principles, such as poorly abstracted code, high coupling between modules, or the presence of infrastructure-specific "code smells" [3].
- **Security Debt**: The accumulation of security-related issues, such as misconfigurations, vulnerable provider versions, or hard-coded secrets exposed in the codebase [4].

If left unmanaged, this debt accumulates, leading to structural erosion that makes the infrastructure brittle, difficult to evolve, and increasingly insecure.

While a variety of industrial tools and academic frameworks exist for vulnerability scanning and code quality measurement, the longitudinal relationship between these two dimensions has

been largely unexplored. In practice, security is often treated as an isolated concern, addressed through point-in-time checks for misconfigurations. The primary objective of this research is to investigate whether the structural quality of Terraform code is empirically associated with security risk. To this end, we introduce **TerraPulse**[1], a framework developed to empirically investigate this connection through a longitudinal analysis of real-world repositories. TerraPulse introduces two synthetic indicators: a **Security Debt Index (SDI)**, which quantifies risk based on vulnerability severity, and a data-driven **Structural Debt Index (StDI)**, which aggregates structural metrics weighted by their empirically derived predictive power.

To guide our empirical investigation, this study aims to answer the following four fundamental Research Questions (RQs):

- **RQ1 (Association):** Is there a statistically significant correlation between structural code metrics and the Security Debt Index (SDI) in Terraform repositories?
- **RQ2 (Prediction):** Which structural attributes are the most reliable predictors for the emergence of security vulnerabilities?
- **RQ3 (Evolution):** How do structural and security debt co-evolve over the lifecycle of an IaC project, and what joint trend patterns can be identified across the population?
- **RQ4 (Characterization):** Is it possible to identify distinct clusters of Terraform projects with homogeneous structural and security debt intensity profiles, and what distinctive characteristics do they present?

The main contributions of this paper are:

- A large-scale, longitudinal empirical study on a statistically representative sample of 500 Terraform repositories, analyzing 10,054 historical snapshots filtered down to 9,259 unique code states to prevent temporal autocorrelation.
- The definition and implementation of two novel indices: a data-driven StDI, weighted by Random Forest feature importance, and an SDI for quantifying security risk in IaC.
- Empirical evidence from a Random Forest model, rigorously validated using GroupKFold to prevent data leakage, that structural metrics are associated with 39% ($R^2 = 0.397$, log-transformed CV) of the variance in security debt on unseen repositories, with the number of resources (`num_resources`) being the dominant predictor (38% importance). Held-out evaluation on the original SDI scale yields $R^2 = 0.133$, reflecting the difficulty of cross-repository generalization for this highly heterogeneous metric.
- A quantitative analysis of evolutionary dynamics on 308 mature repositories, revealing that 74.4% of projects exhibit increasing structural debt over time, while joint improvement of both structural and security debt is exceptionally rare (0.6%), consistent with Lehman's laws of software evolution [11].
- An unsupervised characterization of three distinct project archetypes—*Low-to-Moderate-Debt* (28.4%), *High-Debt* (64.6%), and *Very-High-Debt* (7.0%)—derived from debt intensity features, providing an empirical basis for risk stratification in IaC ecosystems.

The remainder of this paper is organized as follows: Section II reviews related work and theoretical foundations. Section III details the empirical study design. Section IV presents and analyzes the results for each research question. Section V discusses the implications of our findings. Section VI outlines the threats to validity, and Section VII concludes the study with future work.

II. Related Work and Theoretical Background

This study lies at the intersection of Infrastructure as Code (IaC) quality, software security, and mining software repositories (MSR). In this section, we review the theoretical foundations and state-of-the-art research that underpin our work.

A. Software Quality Models for IaC

The assessment of software quality is traditionally governed by the ISO/IEC 25010 standard. While designed for general-purpose software, recent studies have demonstrated its applicability to IaC. Konala *et al.* [5] proposed a formal mapping of IaC attributes to the ISO product quality model, emphasizing that quality in IaC extends beyond syntax correctness to include analyzability and changeability.

The translation of these abstract concepts into quantitative metrics was pioneered by Dalla Palma *et al.* [3], who defined a catalog of 46 metrics for Ansible (e.g., complexity, coupling, size). More recently, Begoug *et al.* introduced *TerraMetrics* [6], a static analysis tool specifically for Terraform that extracts 40 structural metrics from the Abstract Syntax Tree (AST). However, existing research has primarily focused on defining these metrics or validating them against general defects [7]. Our work extends this by explicitly correlating structural metrics not just to general defects, but specifically to *security vulnerabilities*, and by analyzing their evolution over time rather than in static snapshots.

B. Security Smells and Debt in IaC

Parallel to structural quality, the security dimension of IaC has been formalized through the concept of "Security Smells." Rahman *et al.* [4] identified seven recurring patterns ("The Seven Sins") that indicate security weaknesses in Puppet scripts, such as hard-coded secrets and unnecessary administrative privileges.

In the broader context of software engineering, these issues contribute to Technical Debt [1], [2], specifically "Security Debt." While industrial tools like Trivy [10] allow practitioners to detect these vulnerabilities, the academic literature lacks a longitudinal analysis of how this debt accumulates in Terraform projects and whether it is associated with structural

[1]https://github.com/NuclearOX/TerraPulse

degradation. This gap motivates our **RQ3**, which investigates the co-evolution of these two debt dimensions.

C. Mining Software Repositories and Software Evolution

The study of how software changes over time is grounded in the laws of software evolution, which state that systems naturally increase in complexity and entropy unless actively maintained through refactoring [11]. Validating these principles in modern domains requires robust Mining Software Repositories (MSR) techniques. The release of *TerraDS* by Bühler *et al.* [8] provided the first large-scale, curated dataset of Terraform projects, enabling systematic research. Furthermore, tools like *PyDriller* [9] have simplified the extraction of process metrics from Git histories.

TerraPulse builds upon these foundations by integrating the mining capabilities of PyDriller with a hybrid structural analyzer and the security scanning of Trivy. Unlike previous studies that often analyze single snapshots, our framework reconstructs the entire commit history of repositories to characterize both the longitudinal co-evolution of structural and security debt (**RQ1**, **RQ2**, **RQ3**) and the debt intensity profiles of distinct project clusters (**RQ4**).

III. Study Design

To address the research questions, we designed a large-scale longitudinal study targeting the evolution of Terraform projects. The study was conducted using **TerraPulse**, a custom Python-based framework developed to automate the mining, analysis, and metric extraction processes.

A. Dataset Selection and Adaptive Sampling

We selected the **TerraDS** dataset [8] as our data source. TerraDS aggregates metadata and source code for over 67,000 Terraform repositories from GitHub. To ensure the industrial relevance of our findings and exclude "toy" or abandoned projects, we filtered the population to include only active repositories with a star count ≥ 10. This resulted in a relevant population of $N = 2{,}693$ repositories.

Given the high computational cost of longitudinal security scanning, analyzing the entire population was infeasible. We determined the minimum sample size using **Cochran's Formula** [13] with a 95% confidence level ($Z = 1.96$) and a 5% margin of error ($e = 0.05$), yielding a theoretical requirement of 337 repositories. To enhance statistical power, we set a target of **500 successfully analyzed repositories**. Because mining raw Git histories is prone to technical failures (e.g., clone timeouts, missing `.tf` files), TerraPulse implements a dynamic sampling buffer. The framework attempted to mine 513 repositories, seamlessly discarding 13 that failed during the pipeline (a 2.5% technical dropout from the attempted pool), until exactly 500 valid, fully-analyzed repositories were secured.

B. Three-Stage Statistical Validation

To ensure that our final subset of 500 repositories is statistically representative of the entire population, we performed a rigorous three-stage validation using the non-parametric **Kolmogorov-Smirnov (K-S) test** across key metrics like popularity (Stars) and size (KB).

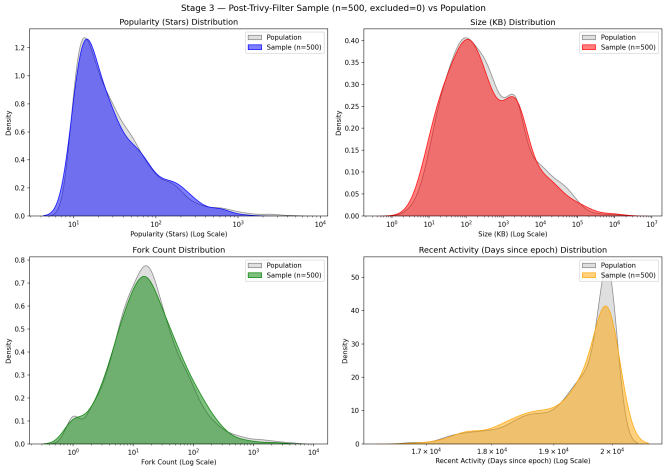

Fig. 1. Kernel Density Estimation (KDE) comparing the distribution of the final filtered sample (blue/red) against the total population (grey) for Popularity and Size. The close overlap confirms statistical representativeness.

1) **Theoretical Validation:** The initial random draw of 500 targets closely mirrored the population ($p > 0.95$).
2) **Empirical (Post-Dropout) Validation:** We tested the 500 successfully mined repositories to ensure the 13 technical dropouts did not introduce a selection bias toward smaller projects. The empirical sample remained highly representative ($p > 0.77$).
3) **Post-Trivy Filter Validation:** Security scanners can produce systematic false negatives if they fail to recognize IaC files. We applied a strict diagnostic filter, excluding any repository where Trivy failed to identify at least one Terraform target across its history. Notably, **0** of the 500 repositories required exclusion at this stage.

As shown in Fig. 1, the distributions of the final sample and the population are closely aligned. We cannot reject the null hypothesis that they originate from the same distribution, supporting the external validity of our findings.

C. Data Collection and Deduplication

TerraPulse implements an *Adaptive Mining* strategy to reconstruct the history of each repository. Since IaC projects often lack consistent semantic versioning, the tool prioritizes Git tags when available; otherwise, it samples up to **50** equidistant commits from the repository's full commit history.

To avoid statistical bias caused by temporal autocorrelation, we implemented a **Content-Hashing Deduplication** mechanism. At each checkout, TerraPulse calculates an MD5 hash exclusively on production `.tf` and `.tfvars` files, automatically pruning non-infrastructural directories (e.g., `tests/`, `.git/`). Snapshots that do not introduce actual changes to the infrastructure code are flagged as duplicates and excluded. This process condensed the **10,054 raw historical snapshots** into a final dataset of **9,259 unique code states**.

D. Variable Definitions

1) Security Debt Index (SDI): Security debt acts as our dependent variable. We employed **Trivy** [10], an industry-standard security scanner, running in a controlled offline container to detect misconfigurations, vulnerabilities in providers, and exposed secrets. The raw occurrences are aggregated into a single risk score using the median severity weights of the **Common Vulnerability Scoring System (CVSS v3.1)** [18]:

$$SDI = \sum_{s \in S} (Count_s \times Weight_s) \tag{1}$$

where $S = \{Critical, High, Medium, Low\}$ and weights are 9.5, 8.0, 5.0, and 2.0, respectively.

2) Structural Debt Index (StDI): Structural debt acts as our independent variable. We adopted an automated, hybrid parsing approach (combining an Abstract Syntax Tree parser via `python-hcl2` with regex-based fallbacks for resilience) to extract **10 core structural metrics**.

Our deliberate selection of this concise metric set, as opposed to the exhaustive catalogs proposed for Configuration Management tools (e.g., the 46 Ansible metrics by Dalla Palma et al. [3]) or general IaC frameworks [5], is guided by four strict methodological constraints:

1) *Paradigm Compatibility*: Terraform is a declarative provisioning tool, rendering procedural, task-based metrics (e.g., `ignore_errors`, `rescue` blocks) inapplicable.
2) *Variable Independence*: We strictly excluded security-specific attributes (e.g., exposed SSH keys, deprecated modules) from the structural metric set to prevent tautological associations with our dependent variable (SDI).
3) *Code vs. Metadata*: We focused solely on intrinsic source code properties, excluding repository metadata (e.g., stars, forks) which reflect popularity rather than engineering quality.
4) *Statistical Parsimony*: To prevent multicollinearity and the curse of dimensionality in our Machine Learning models, we distilled the feature space into orthogonal dimensions aligned with the ISO/IEC 25010 mapping proposed by Konala et al. [5].

These 10 core metrics cover size (lines of code, resource count, variable count, output count), coupling (provider count, module references, internal references), and complexity/maintainability (an approximated IaC-McCabe complexity index[2], hard-coded values, comment lines). To capture intensity independent of project scale, we engineered 4 additional density metrics (complexity density, hard-coded density, comment density, and resource density), resulting in a final feature space of **14 structural attributes**.

Unlike traditional software quality models (e.g., SQALE-based analyzers) or heuristic linters that assign arbitrary, expert-defined remediation weights to quality violations, we adopted a purely **data-driven** method. To construct the StDI, each metric m_i is first standardized via intra-project Z-score normalization. The index is then computed as a weighted sum:

$$StDI = \sum_{i=1}^{n} \big(Z(m_i) \times w_i\big) \tag{2}$$

where w_i represents the Feature Importance coefficient derived from the Random Forest regressor trained in RQ2 (detailed in Section IV-B). This ensures that the StDI represents a structurally grounded measure of risk. We note that this introduces a partial methodological dependency between RQ2 and RQ3, which is discussed in Section VI.

[2]The IaC-McCabe index counts conditional constructs (`count`, `for_each`), dynamic blocks, and ternary expressions—the primary branching constructs available in HCL—as a proxy for cyclomatic complexity.

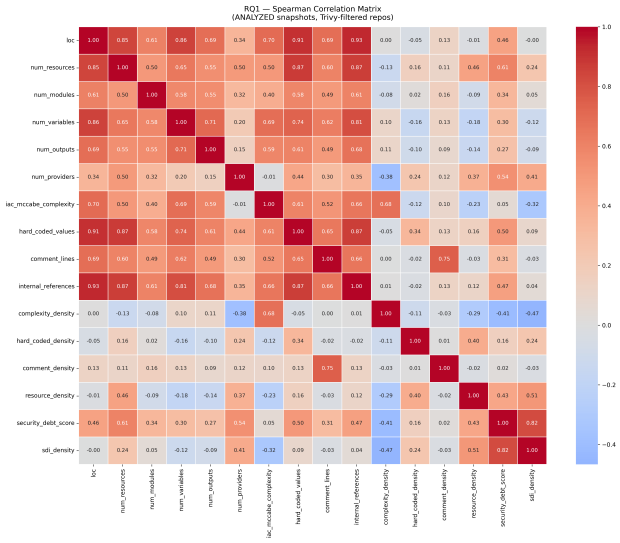

Fig. 2. Spearman Correlation Heatmap (unique code states). Darker red indicates a stronger positive correlation with Security Debt.

IV. Results

In this section, we present the empirical findings for the four research questions. The analysis was performed on the filtered dataset of **9,259 unique code states** (extracted from the 500 validated repositories) to prevent temporal autocorrelation bias resulting from commits that do not alter the actual infrastructure code.

A. RQ1: Association between Structure and Security

The first research question investigates whether structural characteristics of Terraform code are statistically associated with security debt. We approached this using two complementary methods: non-parametric correlation analysis and robust multivariate regression modeling.

1) Correlation Analysis: We calculated the Spearman rank correlation coefficient (ρ) between structural metrics and the Security Debt Index (SDI). To ensure statistical rigor across multiple comparisons, significance was evaluated applying the **Benjamini-Hochberg False Discovery Rate (FDR)** correction [16] ($\alpha = 0.05$). The full spectrum of these relationships is visualized in the heatmap in Fig. 2.

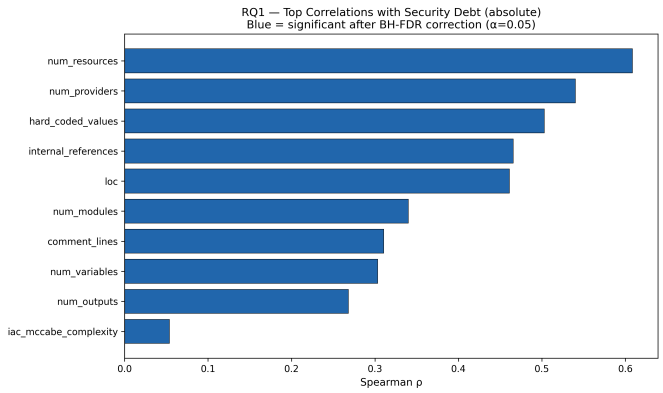

Fig. 3. Top Spearman correlations with absolute Security Debt. Blue bars indicate statistical significance after BH-FDR correction ($\alpha = 0.05$).

The analysis reveals that absolute security debt is most strongly associated with the logical scale and external coupling of the infrastructure. **As highlighted in Fig. 3**, the dominant correlates of SDI are `num_resources` ($\rho = 0.609$), `num_providers` ($\rho = 0.540$), and `hard_coded_values` ($\rho = 0.503$), all highly significant ($p < 0.001$). Lines of code (`loc`) also shows a meaningful association with absolute debt ($\rho = 0.461$), though its independent contribution is substantially absorbed by `num_resources` due to high multicollinearity ($\rho = 0.85$ between the two).

A more nuanced picture emerges when analyzing **debt density** (SDI normalized by lines of code). `loc` becomes statistically **insignificant** ($\rho = -0.003, p = 0.788$), confirming that raw file size is not a driver of per-unit risk. Importantly, `iac_mccabe_complexity` shows a significant *negative* association with debt density ($\rho = -0.319$). This preliminary observation—that complexity is associated with *lower* risk per unit of code—forms the basis of what we call the **"Complexity Paradox,"** explored in greater depth through multivariate regression below.

2) Multivariate Regression (OLS vs. RLM): To isolate the marginal contribution of each metric while controlling for confounding factors, we fitted a multiple linear regression model. To prevent model distortion, we iteratively removed features exhibiting a Variance Inflation Factor (VIF) > 10. Notably, `loc` and `hard_coded_values` were discarded during this phase, as their variance is heavily subsumed by `num_resources` (high multicollinearity).

The surviving predictors were Z-score normalized to ensure coefficient comparability. The Ordinary Least Squares (OLS) model achieved an Adjusted R^2 of **0.580**, indicating that the selected structural attributes explain 58% of the variance in security debt. However, we note that the OLS Durbin-Watson statistic of **0.108** indicates severe within-repository serial autocorrelation in the residuals, which renders OLS standard errors unreliable and inflates apparent significance. For this reason, we treat OLS as exploratory and anchor our theoretical conclusions to the **Robust Linear Model (RLM)** with Huber's T norm, which is resistant to both outliers and distributional violations. Table I compares the two models.

TABLE I
MULTIVARIATE REGRESSION RESULTS (TARGET: SECURITY DEBT SCORE)

Z-Normalized Metric	OLS Coef.	RLM Coef.	P-Value	Impact (RLM)
Intercept	313.68	213.48	< 0.001	Base Debt
`num_resources`	785.24	586.25	< 0.001	**Increases**
`num_variables`	292.77	74.83	< 0.001	**Increases**
`num_modules`	-46.62	21.56	< 0.001	**Increases***
`num_providers`	27.47	15.09	< 0.001	**Increases**
`comment_lines`	-76.41	-33.84	< 0.001	**Decreases**
`num_outputs`	-73.82	-42.93	< 0.001	**Decreases**
`iac_mccabe_complexity`	**-374.91**	**-251.29**	< 0.001	**Decreases**

*Sign flip from OLS(-46.62) to RLM($+21.56$) indicates OLS coefficient was distorted by outliers.

The RLM sign flip for `num_modules` (negative in OLS, positive in RLM) reveals that relying on external modules introduces a slight security overhead when the distorting effect of extreme outliers is removed.

Most importantly, the regression formalizes the **"Complexity Paradox."** This pattern must be interpreted carefully across its three components: (1) `iac_mccabe_complexity` has a weak positive bivariate correlation with *absolute* SDI ($\rho = +0.054$), as larger and more complex projects naturally accumulate more total vulnerabilities; (2) it shows a significant negative correlation with *SDI density* ($\rho = -0.319$), meaning that per unit of code, more complex projects carry lower risk; and (3) controlling for all other metrics, the RLM coefficient is strongly negative (-251.29), indicating a direct protective effect. Taken together, these three observations support the interpretation that Terraform code leveraging logical abstraction constructs (`for_each`, `count`, dynamic blocks) tends to exhibit lower security debt per unit of infrastructure than flat, repetitive code of equivalent scale. This is consistent with the engineering principle that DRY (Don't Repeat Yourself) code reduces the surface area for misconfiguration.

B. RQ2: Predictive Power of Structural Metrics

While RQ1 established correlational associations, RQ2 investigates whether structural metrics can predict the magnitude of security debt across unseen projects. We trained a Random Forest Regressor [17] using 14 features: the 10 raw structural metrics plus four density ratios (`complexity_density`, `hard_coded_density`, `comment_density`, and `resource_density`), which capture intensity independent of project scale.

To handle the extreme right-skewness typical of technical debt distributions, the target variable (SDI) was log-transformed ($\log(1 + x)$) prior to training, with predictions back-transformed for evaluation on the original scale. Crucially, to prevent data leakage and ensure cross-repository generalizability, we employed a **Group-Aware Cross-Validation** strategy (`GroupKFold`, 5 folds) grouped by repository name. This strictly guarantees that snapshots from the same project never appear simultaneously in both training and validation sets.

1) Model Performance: In the log-transformed space, the model achieves a mean cross-validated R^2 **of 0.397** (± 0.045). This score measures the model's ability to capture relative

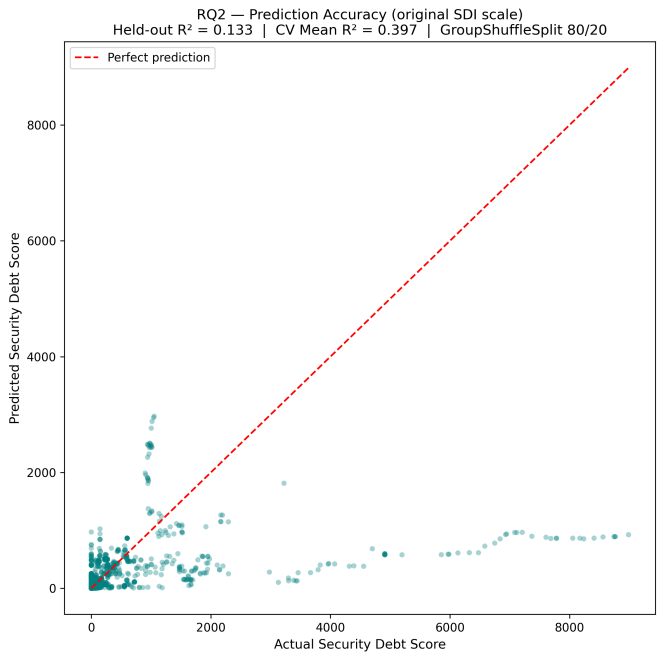

Fig. 4. Prediction accuracy (actual vs. predicted SDI on original scale, held-out test set). The model captures the lower risk range effectively; systematic under-prediction for SDI $>$ 4,000 reflects unmeasured socio-technical factors.

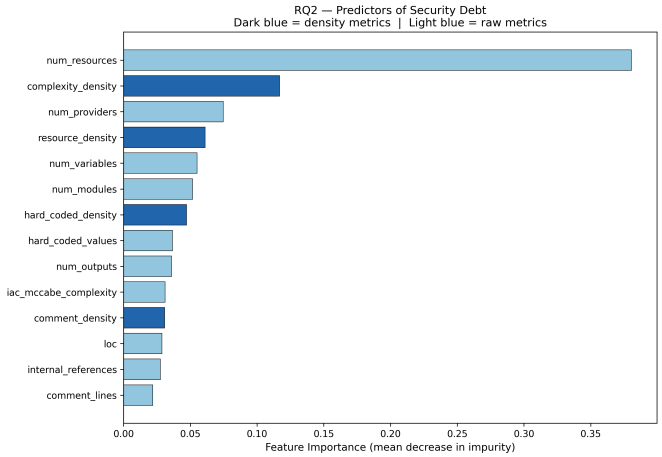

Fig. 5. Feature importance ranking. `num_resources` and `complexity_density` are the dominant predictors, while raw Lines of Code (`loc`) is largely absorbed by more informative features.

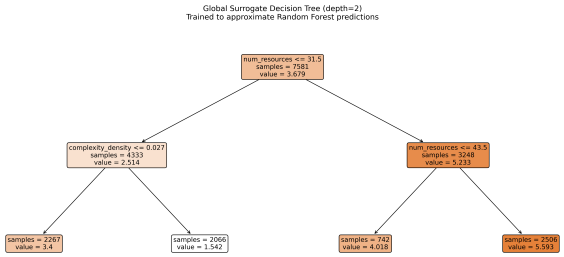

Fig. 6. Global Surrogate Decision Tree (depth $=$ 2, fidelity R^2 $=$ 0.711 against the Random Forest's predictions). This proxy model was trained to approximate the ensemble's output rather than the raw SDI labels, and serves a purely illustrative purpose. The surrogate confirms the dominance of `num_resources` as the primary splitting criterion, while `complexity_density` emerges as the key discriminant for small-scale projects—consistent with the Complexity Paradox identified in RQ1.

rank ordering of security debt across unseen repositories in a compressed representation where distributional variance is smoothed.

When evaluated on a strictly held-out test set (80/20 GroupShuffleSplit, 400 training repositories, 100 test repositories), on the **original SDI scale**, the model achieves an R^2 of **0.133**, a Mean Absolute Error (MAE) of **332.38** points, and an RMSE of **1,042.02**. The gap between the CV R^2 (0.397) and the held-out R^2 (0.133) is expected and informative: the log-transform compresses the variance of outlier repositories during training, while evaluation on the raw scale is dominated by prediction errors on heavily indebted repositories. This indicates that while structural metrics carry a consistent directional signal, the precise magnitude of security debt is influenced by factors beyond the structural features available in this study—such as team practices, update policies, and provider-specific vulnerability exposure.

Fig. 4 visualizes the relationship between actual and predicted SDI on the held-out set. The model captures the lower portion of the risk range well but systematically under-predicts for repositories with SDI above 4,000, confirming that extreme outliers remain a challenge for cross-project generalization.

2) Feature Importance Analysis: The Random Forest model allows us to rank predictors by their contribution to impurity reduction (Gini importance). This ranking, visualized in Fig. 5, provides a clear hierarchy of structural risk factors.

`num_resources` is the overwhelmingly dominant predictor (importance: **0.380**), confirming that the logical footprint of the infrastructure—the number of managed cloud entities—is the primary driver of attack surface and misconfiguration potential. The second-most important predictor is `complexity_density` (importance: **0.118**), reinforcing the Complexity Paradox from RQ1: the concentration of logical abstraction per unit of code carries a distinct and highly informative risk profile. `num_providers` (0.073) and `resource_density` (0.061) rank third and fourth, respectively. Conversely, the raw `loc` metric (importance: **0.029**) ranks near the bottom—not because project size is irrelevant, but because its predictive signal is largely absorbed by `num_resources` ($\rho = 0.85$ between the two), and by the density features, which capture risk intensity more precisely.

To expose the ensemble's decision logic, Fig. 6 presents a **Global Surrogate Decision Tree** (depth $=$ 2, fidelity R^2 $=$ 0.711 against the Random Forest's predictions). This proxy model was trained to approximate the ensemble's output rather than the raw SDI labels, and serves a purely illustrative purpose. The surrogate confirms the dominance of `num_resources` as the primary splitting criterion, while `complexity_density` emerges as the key discriminant for

small-scale projects—consistent with the Complexity Paradox identified in RQ1. For DevSecOps practitioners, these findings jointly underscore that audits should prioritise resource footprint monitoring and abstraction density rather than raw file size.

C. RQ3: Evolutionary Dynamics of Technical Debt

This research question moves beyond static snapshots to investigate how structural and security debt co-evolve over a project's lifecycle. We analyzed a subset of **308 mature repositories** (defined as having at least 5 unique snapshots and at least one non-zero SDI observation) to identify longitudinal patterns.

1) Independent Trend Analysis: We applied the non-parametric **Mann-Kendall test** [14], [15] to each repository's StDI and SDI time series to identify statistically significant monotonic trends. The independent distributions reveal a stark contrast between structural and security evolution:

- **Structural Debt (StDI):** A large **74.4%** of projects exhibit a continuously increasing structural debt. Only 1.9% show a decreasing trend. This provides empirical support for Lehman's laws of software evolution [11], suggesting that without proactive refactoring, structural degradation is the default trajectory for IaC projects.
- **Security Debt (SDI):** Security debt evolution is more varied. While 37.0% of projects exhibit increasing security debt, 45.5% show no significant monotonic trend, and 17.5% show a decreasing trend. This indicates that targeted security interventions (e.g., updating a vulnerable provider) occur more frequently than deep structural refactoring.

We note that the Mann-Kendall test is designed to detect *monotonic* trends; non-monotonic patterns (e.g., debt rising then falling) are classified as "no trend," which may undercount true evolutionary activity in some repositories.

2) Joint Co-evolution Patterns: To understand how these dimensions interact, we cross-tabulated the SDI and StDI trends into a 3×3 contingency matrix, visualized in Fig. 7.

The most prevalent evolutionary pattern is **Co-deterioration** (increasing SDI / increasing StDI), representing **31.5%** of the analyzed repositories. When combined with projects showing increasing StDI but stagnating SDI (29.2%), over 60% of the ecosystem exhibits continuous structural decay accompanied by persistent or worsening security risk. Conversely, joint remediation (decreasing SDI / decreasing StDI) is a statistical anomaly, occurring in only **0.6%** of projects (2 repositories). This is consistent with the notion that technical debt in IaC is "sticky" [2] and rarely fully repaid.

Longitudinal Spearman correlations between StDI and SDI across the 308 repositories show a mean ρ of 0.257 (median: 0.313), but a standard deviation of 0.567, indicating substantial heterogeneity in how the two dimensions co-evolve at the individual project level.

3) Qualitative Analysis of Case Studies: To illustrate these dynamics, we selected representative repositories for each major trend pattern based on the magnitude of their intra-project Spearman correlation between StDI and SDI.

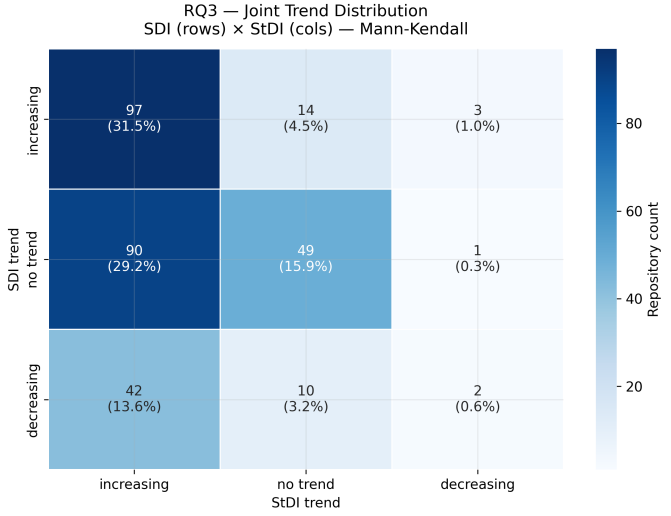

Fig. 7. Joint trend distribution (SDI vs. StDI) based on Mann-Kendall tests across 308 mature repositories. Over 60% of projects experience structural decay accompanied by increasing or stagnant security risk.

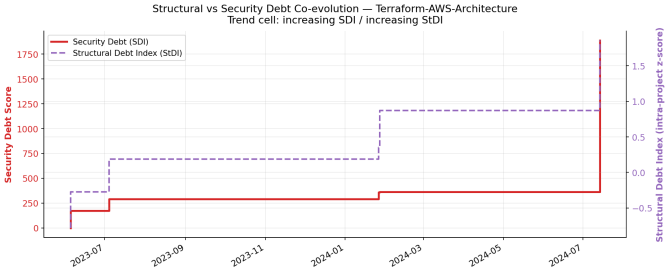

Fig. 8. *Co-deterioration* pattern (`Terraform-AWS-Architecture`). Both structural debt (dashed purple) and security debt (solid red) increase in tandem over approximately 14 months, driven by dependency accumulation as the codebase scales.

Fig. 8 illustrates the dominant **Co-deterioration** pattern. In `Terraform-AWS-Architecture`, both StDI and SDI grow in distinct steps as the infrastructure expands over roughly 14 months. The debt composition chart (Fig. 9) reveals that the post-March 2024 acceleration is driven predominantly by dependency debt (provider CVEs), not by new infrastructure misconfigurations—a compositional shift invisible to purely structural monitoring.

Fig. 10 shows a discordant pattern: **Structural Refactoring without Security Benefit**. Over approximately 13 months, the `hub-and-spoke-with-shared-services-vpc-terraform` project's StDI decreases through refactoring episodes, yet its SDI increases in step-wise jumps. This highlights that structural improvements can occur independently of security remediation—developers who invest in code quality do not automatically eliminate misconfigurations. All debt is exclusively composed of infrastructure misconfigurations (Fig. 11).

Fig. 12 presents the most longitudinally rich case study:

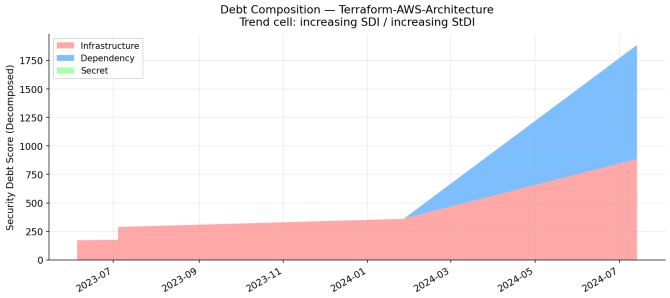

Fig. 9. Debt composition for `Terraform-AWS-Architecture`. The acceleration after early 2024 is driven by dependency (provider CVE) debt, not infrastructure misconfigurations.

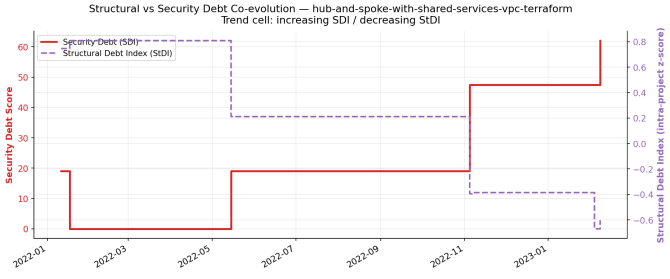

Fig. 10. *Structural Refactoring without Security Benefit* (`hub-and-spoke...`). StDI decreases through refactoring activity, yet SDI continues to increase, demonstrating that structural improvement does not automatically translate to security improvement.

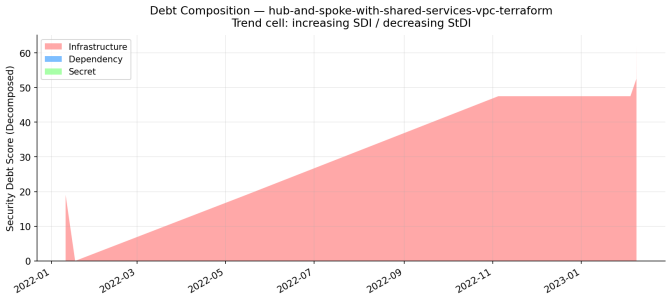

Fig. 11. Debt composition for `hub-and-spoke...`: entirely infrastructure-driven, gradually accumulating over the observation period.

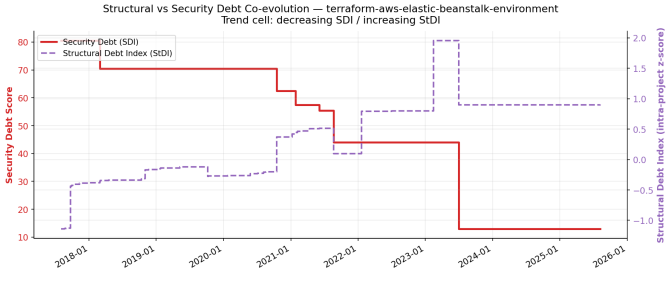

Fig. 12. *Security Remediation with Structural Neglect* (`terraform-aws-elastic-beanstalk-environment`). Over 8 years, SDI is actively and repeatedly reduced while StDI monotonically increases, demonstrating long-term divergence between the two debt dimensions.

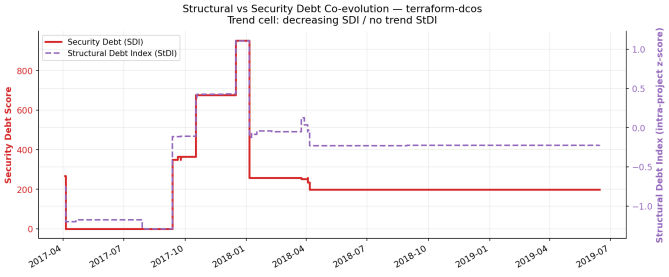

Fig. 13. *Targeted Security Remediation* (`terraform-dcos`). A single large security patch causes a sharp drop in SDI, while StDI shows no monotonic trend. The underlying structural quality is unaffected.

Security Remediation with Structural Neglect. The `terraform-aws-elastic-beanstalk-environment` repository spans nearly 8 years (2018–2026) and shows a clear pattern of active, repeated security remediation—SDI is progressively reduced from $\approx$ 72 to $\approx$ 13—while StDI monotonically increases throughout. This long-term divergence between the two dimensions is the clearest empirical illustration of our central finding: security and structural quality are distinct, independently managed concerns. A team that diligently patches vulnerabilities does not thereby improve structural quality, and vice versa.

Finally, Fig. 13 depicts **Targeted Security Remediation**. In `terraform-dcos`, the SDI drops sharply following a specific commit in early 2018, indicating a focused security intervention. The StDI shows no statistically significant monotonic trend across the observation period. This represents the common industry approach: reactive ”scan-and-fix” patching that addresses the immediate vulnerability without improving the underlying structural health of the codebase.

D. RQ4: Characterization of Project Archetypes

The final research question investigates whether distinct archetypes of Terraform projects exist based on their structural and security debt profiles. We applied an unsupervised K-Means clustering algorithm on the latest snapshot of all 500 evaluated repositories.

1) Methodology and K Selection: To prevent project scale from dominating the clustering space, we engineered a feature space based on **debt-intensity ratios** (e.g., SDI per LOC, complexity per resource) and debt composition shares (infrastructure, dependency, and secret share). To mitigate artefactual outliers from near-zero denominators, we applied **95th-percentile winsorization** to all ratio features, followed by $\log(1 + x)$ transformation and Z-score standardization.

We evaluated K-Means for $K \in [2, 8]$. As shown in Fig. 14, the elbow method shows an inflection at $K = 3$, and the Silhouette Score reaches its global maximum at $K = 3$ (0.4117). We note that $K = 4$ yields a nearly identical silhouette score (0.4115, $\Delta = 0.0002$); the choice of $K = 3$ is thus supported by the combination of the elbow criterion and parsimony, though an alternative four-cluster solution cannot be ruled out on statistical grounds alone.

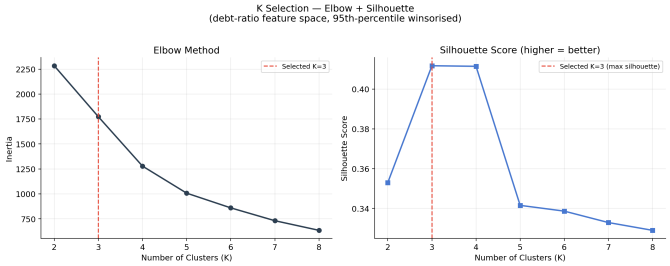

Fig. 14. K selection using the Elbow method (left) and Silhouette Score (right). $K = 3$ is selected at the maximum silhouette; $K = 4$ yields a nearly identical score ($\Delta = 0.0002$).

TABLE II
PROJECT ARCHETYPE CHARACTERISTICS (MEAN VALUES)

Archetype	Count (%)	Avg. LOC	Avg. SDI	SDI/LOC
Low-to-Moderate-Debt	142 (28.4%)	1,297	2.5	0.006
High-Debt	323 (64.6%)	2,025	347.3	0.261
Very-High-Debt	35 (7.0%)	3,032	906.1	0.605

2) Archetype Analysis: The clusters were dynamically labeled based on their relative security debt intensity (`sdi_per_loc`). Table II summarizes the mean characteristics of each archetype.

The resulting archetypes reveal a highly non-uniform distribution of risk across the Terraform ecosystem:

- **Low-to-Moderate-Debt (28.4%):** The "Virtuous Modules." These projects maintain a near-zero security debt (mean SDI $= 2.5$). We note that this cluster is predominantly composed of repositories with *zero* observed security debt throughout their history; the label "low-to-moderate" describes the cluster boundary rather than a continuously distributed profile. Their defining structural trait is a very high complexity-per-resource ratio (mean: 4.05), compared to 0.85 for the High-Debt cluster. This provides cross-sectional support for the Complexity Paradox identified in RQ1: repositories that heavily utilize logical abstraction tend to be the most secure.
- **High-Debt (64.6%):** The "Statistical Norm." Representing the majority of the ecosystem, these projects exhibit significant security debt (mean SDI $= 347.3$). As visualized in the radar chart (Fig. 15), their debt profile is overwhelmingly dominated by the *infrastructure share* ($\approx 99\%$), indicating that their risk stems primarily from infrastructure misconfigurations accrued during organic growth.
- **Very-High-Debt (7.0%):** The "Systemic Liabilities." While the smallest cluster, they carry the highest average SDI (906.1). Their risk is heavily driven by the *dependency share* ($\approx 64\%$), suggesting reliance on external modules or providers with known CVEs, compounded by a lack of supply-chain governance.

The multidimensional separation of these archetypes is further visualized in Fig. 16 and Fig. 17. Security risk in IaC is not a mere byproduct of scale; it crystallizes into

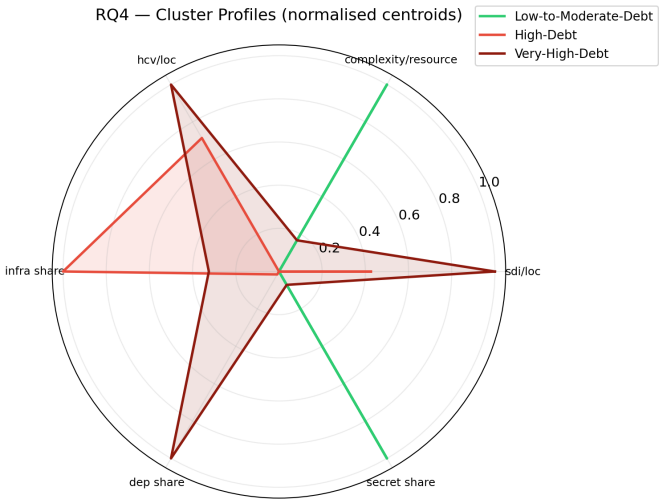

Fig. 15. Radar chart of normalized cluster centroids. *High-Debt* is driven by infrastructure misconfigurations; *Very-High-Debt* is dominated by vulnerable dependencies; *Low-to-Moderate-Debt* is distinguished by high complexity-per-resource.

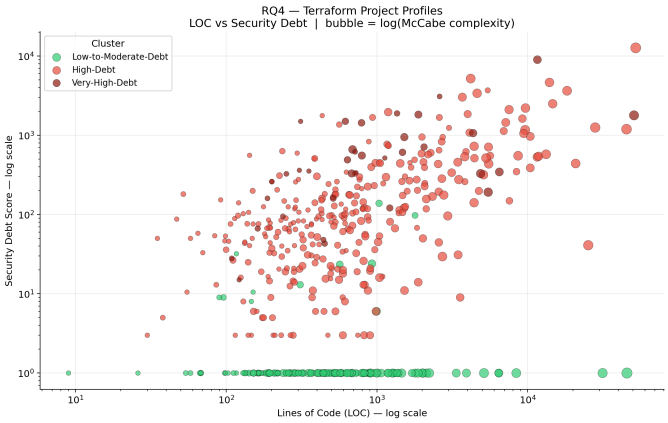

Fig. 16. Scatter plot of Terraform projects (log-log scale). The *Low-to-Moderate-Debt* cluster concentrates along the zero-debt axis; the *Very-High-Debt* cluster separates at the upper risk range.

distinct profiles shaped by engineering habits: abstraction tends to mitigate risk, unmanaged growth is associated with misconfigurations, and neglected external dependencies are associated with the highest severity liabilities.

V. DISCUSSION

The results of our longitudinal study provide a multi-dimensional empirical view of how structural quality and security co-evolve in Terraform projects. In this section, we discuss the broader implications of our findings for practitioners and researchers.

A. Structural Quality as an Associative Signal for Security Risk

One of the central findings of this research is that structural metrics carry a consistent empirical association with security risk. While the cross-validated R^2 of 0.39 (in log-transformed

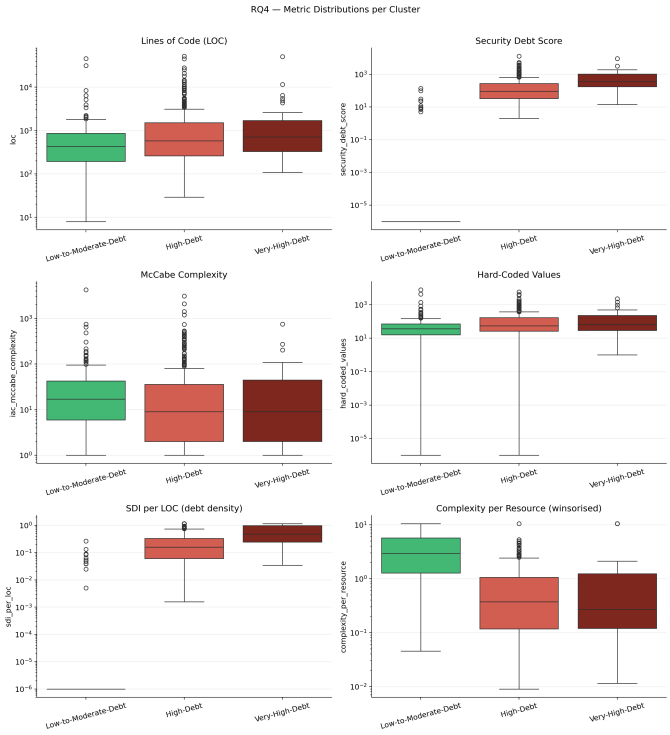

Fig. 17. Metric distributions per cluster. *Low-to-Moderate-Debt* projects exhibit higher complexity-per-resource compared to riskier clusters, consistent with the Complexity Paradox.

space) and the held-out R^2 of 0.133 (on the original scale) reflect the difficulty of cross-repository generalization, they demonstrate a robust underlying pattern: the *way* code is architected is associated with its security posture. Traditionally, IaC security is treated through reactive scanning; our study suggests that structural signals may serve as complementary leading indicators.

The dominance of `num_resources` (38% feature importance) confirms that the logical footprint of the infrastructure is strongly associated with security debt accumulation. The Complexity Paradox—negative correlation of the approximated IaC-McCabe index with debt density ($\rho = -0.319$) and strong protective coefficient in multivariate regression (RLM: -251.29), despite a negligible bivariate correlation with absolute SDI ($\rho = +0.054$)—suggests that modular, logically abstract code is more resistant to debt accumulation per unit of infrastructure. Mature engineering practices characterized by high abstraction (DRY principles) appear to serve as a structural protective factor against the configuration errors that proliferate in flat, repetitive codebases.

B. The Persistence of Debt and Two Distinct Remediation Patterns

The longitudinal analysis (RQ3) reveals two structurally different failure modes in IaC maintenance. The first, and more common, is **co-deterioration** (31.5% of projects): structural and security debt accumulate together, neither dimension being actively managed. The second is **targeted security patching without structural improvement** (17.5% of projects show decreasing SDI, but only 1.9% show decreasing StDI): teams address vulnerabilities reactively but do not invest in the underlying structural quality that produces them. The third, rarest pattern—**joint remediation** (0.6%)—requires simultaneous investment in both dimensions. This extreme rarity is consistent with the ”sticky” nature of technical debt described by Avgeriou *et al.* [2] and with Lehman’s second law of increasing entropy [11]. Practitioners still tend to treat IaC as a one-time deployment artifact rather than a living software system requiring continuous maintenance.

C. Actionable Implications for DevSecOps

Our archetype characterization (RQ4) and predictive models (RQ2) suggest several data-backed directions for practitioners:

- **Prioritize resource footprint over line count:** The near-zero predictive power of `loc` relative to `num_resources` indicates that teams should monitor the number of managed cloud entities as a primary risk proxy, rather than file size. Audits should focus on resource density and abstraction patterns.
- **Consider structural early-warning signals:** A sustained increase in a project’s StDI—even before any new CVE is detected—may indicate increasing risk exposure. Integrating structural debt monitoring into CI/CD pipelines could help intercept accumulating risk before it manifests as exploitable vulnerabilities. We caution, however, that the directional signal is consistent but not a deterministic predictor of near-term security events.
- **Enforce supply-chain governance for large projects:** The Very-High-Debt archetype is characterized not by internal misconfigurations, but by dependency debt (64% of SDI). For projects at this scale, internal scanning of `.tf` files is insufficient; teams must pin provider versions and systematically audit third-party Terraform modules.

VI. Threats to Validity

As with any empirical investigation, our study is subject to threats to validity that must be acknowledged alongside our mitigations.

A. Internal Validity

A primary threat lies in the use of **Trivy** [10] to define our ground truth for security debt. Trivy, like any static scanner, is susceptible to false positives and may miss vulnerabilities not covered by its rule set. To mitigate systematic false negatives, we confirmed that all 500 retained repositories yielded valid Terraform scan targets.

Temporal autocorrelation within repository histories represents a structural threat. Our MD5-based deduplication reduced 10,054 raw snapshots to 9,259 unique code states, mitigating snapshot-level autocorrelation. However, the OLS Durbin-Watson statistic of **0.108** indicates severe within-repository serial correlation in residuals, which inflates OLS significance artificially. All primary theoretical conclusions are therefore anchored to the RLM results and non-parametric

Spearman correlations (BH-FDR corrected), both of which are robust to this condition.

The **StDI is constructed using feature importances derived in RQ2**, which are then applied to longitudinal analysis in RQ3. This partial methodological dependency means that RQ3 trend results partially reflect the RQ2 model's view of structural relevance. Independent weighting strategies (e.g., expert-based or PCA-derived) would be needed to fully decouple these two research questions.

B. External Validity

Our sampling strategy was validated via K-S tests against the TerraDS population, supporting statistical representativeness. However, our population filter ($\geq$ 10 GitHub stars) introduces a selection bias toward better-maintained public repositories. The structural decay and debt persistence observed in RQ3 may be even more pronounced in less visible, private, or corporate environments.

Regarding generalization of the predictive model (RQ2), the gap between the CV R^2 (0.397, log-space) and the held-out R^2 (0.133, original scale) is deliberate and informative. The cross-validated score reflects the model's signal in a variance-compressed representation; the held-out score reflects its operational accuracy on unseen repositories with the full distributional complexity of real-world security debt. We report both to provide a complete picture; practitioners should treat the held-out evaluation as the more operationally relevant metric.

C. Construct Validity

The IaC-McCabe complexity index is an approximation that counts `count`, `for_each`, `dynamic` block, and ternary expressions—the primary branching constructs in HCL—as a proxy for cyclomatic complexity. It does not capture nested module complexity or provider-specific logic. Comparisons with traditional McCabe complexity values should be made with caution.

To prevent scale from distorting the clustering analysis (RQ4), we applied 95th-percentile winsorization on ratio features. This compresses the upper tail of the distribution, which slightly limits the precision of the Very-High-Debt archetype boundary ($n = 35$, 7.0%). The Mann-Kendall test, while appropriate for detecting monotonic trends, classifies non-monotonic patterns (e.g., debt rising then falling) as "no trend", which may undercount genuine evolutionary activity in a subset of repositories.

Case studies in RQ3 were selected based on the magnitude of intra-project Spearman $|\rho|$ between StDI and SDI. This selection criterion favors repositories with strong or consistent directional co-evolution and may not be representative of average behavior within each trend cell.

VII. Conclusion and Future Work

In this paper, we presented a large-scale empirical investigation into the longitudinal relationship between structural code quality and security in the Terraform ecosystem. Through the development of the **TerraPulse** framework, we mined and analyzed the evolution of 500 representative repositories, processing 10,054 historical snapshots to extract **9,259 unique code states**. Our study provides empirical evidence that the structural integrity of Infrastructure as Code is associated with its security posture—not merely a maintainability concern in isolation.

Our findings yield four primary conclusions. First, structural metrics are meaningfully associated with security risk: a cross-validated R^2 of 0.397 (log-space) and R^2 of 0.133 (original scale, held-out) indicate a consistent directional signal that does not fully generalize across the high variance of open-source IaC. Second, the Complexity Paradox holds across all three analytical lenses—bivariate density correlation, multivariate regression, and clustering—supporting the interpretation that logical abstraction is a structural protective factor. Third, our longitudinal analysis of 308 mature projects reveals a systemic pattern of structural decay (74.4% of projects) and security debt persistence; joint remediation of both dimensions simultaneously is exceptional (0.6%), confirming that IaC debt is sticky. Finally, three distinct project archetypes emerge from debt-intensity clustering, with the highest-risk class (Very-High-Debt, 7.0%) primarily threatened by supply-chain vulnerabilities rather than internal misconfigurations.

For practitioners, these findings suggest a shift toward "engineered security" in the cloud: monitoring structural debt alongside vulnerability scanning, and enforcing supply-chain governance for large-scale deployments, may intercept risk accumulation before it manifests as exploitable weaknesses.

A. Future Directions

- **Socio-Technical Analysis (Community Smells):** Future work should investigate whether organizational factors—such as the "Community Smells" defined by Tamburri *et al.* [12]—act as catalysts for the structural degradation observed in RQ3, providing a more holistic view of IaC debt accumulation.
- **Predict-to-Fix and Automated Refactoring:** Leveraging the predictive associations identified here, future tools could move beyond reporting vulnerabilities to suggesting targeted structural refactorings—prioritizing modularization of high-resource, low-abstraction blocks based on their estimated impact on security debt reduction.
- **Cross-Technology Validation:** The TerraPulse methodology is extensible to other IaC technologies (Pulumi, Ansible, AWS CloudFormation). Future work will examine whether the structural predictors identified here—particularly the protective role of abstraction—generalize across the broader IaC ecosystem.

References

[1] W. Cunningham, "The WyCash portfolio management system," in *Addendum to the Proc. of OOPSLA*, 1992.

[2] P. Avgeriou *et al.*, "Managing Technical Debt in Software Engineering (Dagstuhl Seminar 16162)," *Dagstuhl Reports*, vol. 6, no. 4, 2016.

[3] S. Dalla Palma *et al.*, "Toward a catalog of software quality metrics for infrastructure code," *Journal of Systems and Software*, vol. 170, 2020.

[4] A. Rahman *et al.*, "The Seven Sins: Security Smells in Infrastructure as Code Scripts," in *Proc. of ICSE*, 2019.

[5] P. R. R. Konala *et al.*, "A Framework for Measuring the Quality of IaC Scripts," *arXiv:2502.03127*, 2025.

[6] M. Begoug *et al.*, "TerraMetrics: An Open Source Tool for IaC Quality Metrics in Terraform," in *Proc. of ICPC*, 2024.

[7] A. Rahman and L. Williams, "Source Code Properties of Defective Infrastructure as Code Scripts," *Information and Software Technology*, vol. 112, pp. 148–163, 2018.

[8] C. Bühler *et al.*, "TerraDS: A Dataset for Terraform HCL Programs," *arXiv:2407.02906*, 2024.

[9] D. Spadini *et al.*, "PyDriller: Python Framework for Mining Software Repositories," in *Proc. of ESEC/FSE*, pp. 908–911, 2018.

[10] Aqua Security, "Trivy: Vulnerability Scanner for Containers and other Artifacts," https://trivy.dev, accessed Mar. 2026.

[11] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proc. IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980.

[12] D. A. Tamburri *et al.*, "Exploring Social Debt in Software Engineering," in *Proc. of CHASE*, 2015.

[13] W. G. Cochran, *Sampling Techniques*, 3rd ed. John Wiley & Sons, 1977.

[14] H. B. Mann, "Nonparametric Tests Against Trend," *Econometrica*, vol. 13, no. 3, pp. 245–259, 1945.

[15] M. G. Kendall, *Rank Correlation Methods*, 4th ed. Griffin, 1975.

[16] Y. Benjamini and Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society, Series B*, vol. 57, no. 1, pp. 289–300, 1995.

[17] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[18] FIRST.org, "Common Vulnerability Scoring System v3.1 Specification," https://www.first.org/cvss/v3.1/specification-document, 2019.